

Forge

Structural Search and Certified Theory Cooperation Beyond grind

A concrete tactic architecture, exact certificate algorithms, executable prototypes, and a reproducible evaluation package

Central recommendation. Keep grind as a strong ground-reasoning engine. Add a proof-producing layer that can choose induction motives, generalize changing parameters, synthesize and prove missing lemmas, construct witnesses, and commission independently replayable specialist certificates. The central data structure is a scoped graph of proof obligations, not a larger unstructured list of tactics.

What was executed. Python implementations of structural induction and lemma synthesis, exact polynomial certificates, rational Bernstein subdivision, Boolean-integer difference-logic CDCL(T), and residue witnesses. The final test suite has 64 passing tests; 1,238 saved successful proof/model artifacts replay independently of numerical search.

What is not claimed. A finished Lean implementation, Lean-kernel verification of the Python checkers, or measured superiority over grind. Lean was not installed in the execution environment. The included Lean examples and benchmark harness are explicitly uncompiled/unexecuted integration material.

14 September 2026

Documentation baseline: Lean 4.34.0 release and current sources inspected on this date. Numerical library versions and all experiment results are recorded in the archive.

Abstract

A substantially more capable automatic Lean tactic should expand the kinds of proof plans it can construct, not merely increase saturation limits. This article specifies **Forge**: a kernel-oriented architecture above and alongside **grind**, with structural proof search, dependency-correct induction generalization, schema-driven auxiliary-lemma synthesis, scope-correct witness generation, and certificate-producing arithmetic and Boolean engines. We explain the concrete interfaces, scheduling policy, reconstruction obligations, failure modes, and implementation sequence. Four executable prototype families test the central mechanisms. Their outputs include explicit induction traces, exact rational polynomial identities, covering Bernstein trees, resolution proofs with checked difference-logic lemmas, and finite residue tables with a universal soundness argument. Deliberate failures, independent oracles, and certificate mutations are included. The evidence supports feasibility of these components, while leaving the claimed improvement in end-to-end Lean proof rate as a clearly specified experiment rather than an invented result.

Contents

1	The objective: stronger proof construction, not a bigger timeout	4
1.1	What “more powerful” should mean	4
1.2	The deliverable and its evidence levels	4
1.3	A preservation guarantee with an honest resource qualification	4
2	The actual baseline in September 2026	5
2.1	Do not add capabilities that grind already has	5
2.2	Existing tools are implementation assets and strong baselines	6
2.3	Version policy	6
3	Architecture: a scoped graph of obligations	6
3.1	The core objects	6
3.2	Typed sharing and transaction boundaries	7
3.3	Conditionals are proof obligations, not optimistic rewrites	7
3.4	The concrete grind adapter	8
3.5	A concrete scheduling policy	8
4	Structural search: induction that strengthens the right statement	9
4.1	Why generalization is a first-class action	9
4.2	The general Lean algorithm	9
4.3	The step case, with every dependency visible	10
4.4	Schema-driven lemma synthesis	10
4.5	Proof traces, not search replay	10
4.6	Where this reaches beyond ground reasoning	11

5	Nonlinear arithmetic: exact search certificates, not floating-point proofs	11
5.1	The gap to target	11
5.2	The certificate language	12
5.3	An exact and complete quadratic subsolver	12
5.4	A concrete sparse-cone search	13
5.5	Concrete identities returned by the design	14
5.6	Bernstein certificates on rational boxes	14
5.7	Why complementarity matters	15
5.8	Lean reconstruction and semantics	15
6	Boolean search with checkable theory learning	15
6.1	A separate lane before a core-engine rewrite	15
6.2	Difference logic and the integer-negation trap	16
6.3	The certificate rules	16
6.4	What production Lean integration must add	17
6.5	Resource behavior	17
7	Witness synthesis with a universal certificate	18
7.1	Witnesses are scoped terms	18
7.2	An implemented residue family	18
7.3	A sharper synthesis algorithm for the same family	18
7.4	General synthesis proposals	19
8	Retrieval, instantiation, and cooperation that avoid saturation blowups	19
8.1	Retrieve for the missing obligation	19
8.2	Backward premises and forward consequences share one queue	19
8.3	E-matching memoization with the right invalidation rules	19
8.4	Selective alternative forms	20
8.5	An end-to-end worked interaction	20
9	Proof acceptance and the trusted boundary	20
9.1	The kernel must see the original theorem	20
9.2	Axiom auditing must follow the actual Lean version	21
9.3	Malformed or hostile artifacts	21
9.4	Provenance, proof minimization, and export	22

10 Executed experiments and their limitations	22
10.1 Environment and reproducibility	22
10.2 Results by family	22
10.3 The structural ablation	23
10.4 Boolean certificates and independent oracle checks	23
10.5 Timing and size measurements	23
10.6 Mutation tests and negative controls	24
10.7 What the evidence does and does not establish	24
11 The real Lean evaluation that should decide success	24
11.1 Strong baselines and controlled inputs	24
11.2 Benchmark families	25
11.3 Measurements and acceptance criteria	25
11.4 The supplied harness	25
12 An implementable development plan	26
12.1 Module boundaries and order of work	26
12.2 Concrete implementation assets	26
12.3 Proposed user-facing behavior	27
12.4 The most important engineering risks	27
13 Conclusions	27
A Archive map and reproduction commands	28
B A minimal executable certificate example	29
C Scope ledger	29

1 The objective: stronger proof construction, not a bigger timeout

1.1 What “more powerful” should mean

There are three different claims that are easily confused. *Logical strength* concerns what Lean can prove from its axioms; a tactic should not change that. *Search expressiveness* concerns which proof plans the tactic can construct. *Practical strength* concerns solved goals, latency, memory, proof size, and robustness at a specified resource budget. *Forget* targets the second and third, while preserving the first.

My recommendation is a concrete system with five cooperating capabilities:

1. A protected grind lane, plus typed retrieval and bounded structural AND/OR search.
2. Induction with dependency-aware generalization and independently proved auxiliary lemmas.
3. Witness synthesis that respects quantifier order, arithmetic semantics, and dependent types.
4. Exact nonlinear certificates, including rational quadratic decomposition, sparse cones with equality multipliers, and Bernstein subdivision on rational boxes.
5. A proof-producing Boolean/theory lane with learned clauses, explicit theory explanations, and certified reconstruction.

These mechanisms are not five unrelated final tactics. Their outputs become reusable, scoped facts or sub-proofs in one proof-obligation graph. For example, an induction step can request an associativity lemma; its side condition can request an arithmetic fact; a specialist can return a short certificate; the structural step can then finish using grind.

1.2 The deliverable and its evidence levels

The archive contains runnable Python code, exact and object-level checkers, mutation tests, generated certificates, experiment data, prospective Lean examples, and a Lean benchmark harness. The experimental code is deliberately small enough to inspect. It does not pretend to reimplement the dependent type theory of Lean.

Throughout, *implemented* means present in the Python prototype and executed. *mathematically justified* means a soundness argument is given here. *proposed* means an engineering design not yet implemented in Lean. *kernel checked* is reserved for an actual Lean kernel run; no such run occurred in this work. Passing Python replay is useful evidence against implementation mistakes, not a substitute for a proved Lean checker or kernel-accepted proof term.

1.3 A preservation guarantee with an honest resource qualification

A useful default mode first runs stock grind on a pristine state with the user’s original options and a reserved budget B_g . Only if that lane fails does it spend a separate budget B_f on extensions. A faster interleaved mode can be offered, but should not carry the same regression guarantee.

Proposition 1.1 (Protected-lane preservation). *Assume the preserved lane receives the same initial environment, elaborated goal, local context, options, deterministic search choices, and effective resource bounds as the baseline run. If that baseline produces a proof, the protected-lane strategy also produces that proof, provided it is allowed to complete the lane.*

Proof. The preserved computation is an unchanged baseline computation on unchanged inputs. Its successful proof is immediately accepted and returned; no later intervention is required. $\square$

This is not a theorem that a richer tactic dominates at the *same total* time or memory budget. Reserving the full original run and also doing new work costs additional resources. Concurrent scheduling can change effective bounds through memory pressure or timing; therefore the strongest default guarantee uses a serial pristine run, not a shared mutable state.

Failure-triggered interventions have already been explored specifically for `grind`; recent work contrasts them with always-on policies that can disturb successful searches [9]. Consequently, neither preserving the baseline nor adding lookahead is presented here as a novel discovery. The proposed advance is the specific combination of structural goal creation, certified specialist work, and cost-aware reconstruction.

2 The actual baseline in September 2026

2.1 Do not add capabilities that `grind` already has

The current reference describes `grind` as a cooperating reasoning system: congruence closure, propagation, E-matching, guided case analysis, and satellite solvers exchange consequences. It normally reasons by contradiction and constructs Lean proofs. That is already a much stronger baseline than “a simplifier plus linear arithmetic” [1].

Its algebraic component uses polynomial completion/Gröbner-style reasoning for commutative algebra and supports ring and field settings; simply proposing polynomial normalization or a Gröbner basis would not be a meaningful extension [2]. Its linear arithmetic machinery also has its own documented domain and cooperation model [3]. E-matching already has explicit patterns, controls, and generation limits [4]. Case analysis and extensionality-related reasoning must likewise be distinguished from a new general induction planner [5].

Area	Baseline to preserve	Additional Forge work
Equality and algebra	Congruence, instantiated equalities, existing algebraic solver	Goal-directed choice of equivalent forms; certificate-driven inequalities, not a replacement equality engine.
Quantified facts	Pattern-guided theorem instantiation	Backward obligations for missing premises and proved synthesized lemmas.
Datatypes	Constructor reasoning and case splits	Choosing recursors, strengthening induction motives, and proving auxiliary laws.
Existentials	Existing logical propagation	Constructing arithmetic or structured witnesses with explicit scope constraints.
Boolean structure	Existing propagation and splitting	A separately certified CDCL(T) lane for selected large Boolean abstractions.
Automation ecosystem	Existing specialist tactics and premise selectors	A shared proof graph with reconstruction budgets and consistent artifact export.

A constructor case split and an induction principle are not interchangeable. The former tells us that a list is empty or a cons; the latter licenses a recursive hypothesis about its tail. Likewise, recognizing an existential

proposition is not the same as finding a useful witness for it.

2.2 Existing tools are implementation assets and strong baselines

Aesop already supplies structured proof search with rule-driven branching [10]. LeanHammer combines premise selection, Lean-auto translation, and proof-producing tools including Duper and Aesop [11]. Duper provides a higher-order superposition route, while Lean-auto addresses translation from Lean’s dependent setting [12, 13]. LeanSMT supplies another relevant route through SMT solving and reconstruction [14]. These should be reused or measured, not reinvented under new names.

For inequality leaves, Mathlib’s `linarith/nlinarith` infrastructure is an important first choice; the nonlinear preprocessing and certificate-reconstruction architecture make it an appropriate low-cost baseline before a more elaborate search [15, 16]. Existing bit-vector automation is also relevant to a Boolean certificate backend, rather than justification for writing a production SAT solver from scratch [17].

One current compatibility warning matters: the `lean-egg` frontend is marked deprecated in favor of `grind` in its registry documentation [18]. Equality saturation remains a useful technique, but that deprecated frontend should not be recommended as a fresh central dependency. This says nothing against all implementations of e-graphs or the Rust `egg` library.

2.3 Version policy

The release information inspected on 14 September 2026 identified Lean 4.34.0, published that day [6]. This is a *research source baseline*, not a tested compiler environment. The repository’s development branch and the moving `latest` manual are not immutable experimental identifiers.

A production implementation must pin Lean, Mathlib, optional solvers, and the source revision of each adapter. An ecosystem package targeting another Lean version belongs in a separate compatibility matrix until tested. Do not assume that the latest versions of all the libraries mentioned here can be imported into one project. The archive intentionally contains no fabricated Lake lockfile or claimed successful Lean build.

3 Architecture: a scoped graph of obligations

3.1 The core objects

A search node represents a genuine Lean goal together with its scope, not just its printed text. A proposed internal record is:

```

GoalNode = {
  goalMVar, environmentStamp, localContextStamp,
  metavariableSnapshot, universeSnapshot, optionsStamp,
  alternatives : Array ProofPlan,
  dependencies : Array GoalNodeId,
  resourceLedger, status
}
ProofPlan = {
  children : Array GoalNodeId,
  builder : (checked child proofs) -> candidate Lean Expr,
  assumptionsUsed, certificateRef, estimatedReplayCost
}

```

This is interface pseudocode, not a claim that these records already exist in Lean. An OR node offers alternative plans. An AND edge requires every child obligation of a chosen plan to be proved. Shared children may be reused only when their contexts and statements match with the appropriate proven renaming or transport.

Examples of plans are application of a theorem with missing premises, a complete recursor application, witness construction followed by its predicate, and a theory certificate plus its side-condition proofs. A solver returning a candidate model is not a proof plan that closes a Lean goal.

3.2 Typed sharing and transaction boundaries

A node key should include a structural representation of the elaborated expression, free-variable identities and types, universe constraints, relevant instances, transparency policy, and environment revision. Alpha-renaming can be normalized, but dependency edges must remain explicit. Two terms with the same printed name are not necessarily the same variable or constant.

Every speculative branch uses a saved metavariable context. Failed unification, unsuccessful theorem application, or partially constructed induction motives must be rolled back. A cache entry containing unresolved metavariables is either branch-local or parameterized by a checked substitution. Caching a proposition by string and reusing an old proof under new typeclass instances is not acceptable.

The ground equality engine should continue to use typed expressions. Heterogeneous equality and dependent transport require actual proof constructors. Do not flatten a dependent expression into an untyped first-order term and later assume its equivalence classes can be read back without justification.

3.3 Conditionals are proof obligations, not optimistic rewrites

Suppose a retrieved theorem has type $G(t) \rightarrow f(t) = g(t)$. The planner creates an obligation for $G(t)$ and a continuation that can use the equality *after* that obligation is proved. The equality cannot be inserted into congruence closure while its guard is merely plausible.

This matters for cycles. A search that rewrites using $G \rightarrow E$ and then proves G only because it assumed E has not discharged the guard. The proof graph must be acyclic at the level of admitted facts, except for genuine recursor or well-founded induction rules whose cyclic-looking dependencies are justified by the kernel. A candidate lemma remains in a separate conjecture table until its proof has been completed.

3.4 The concrete grind adapter

The inspected API exposes a registration function, `registerSolverExtension`, in the `Lean.Meta.Grind` namespace, together with `SolverExtension.setMethods`, with callbacks `internalize`, `newEq`, `newDiseq`, `mbtc`, `action`, `check`, and `checkInv`. Registration is initialization-time, and extension state is per goal. The API also distinguishes `Solvers.mkActionCore` from the wrapper that drains pending facts [7]. These are real integration points, not hypothetical hooks.

A polynomial adapter can mark relevant expressions during internalization, mark a component dirty after an equality, and do bounded certificate search on the expensive-check path. Its accepted output is a proved fact, published using the ordinary fact machinery. The main implementation’s parameter structure also has explicit extra facts and theorem data, which supports an outer adapter that does not immediately modify the core [8].

There are two boundaries that should remain explicit. First, structural induction should operate on the original goal or a suitable outer goal state, not accidentally on a fully transformed contradiction state where the useful motive has been obscured. Second, replacing the entire Boolean branch manager is not achieved by installing a solver callback. Start with an independent certificate-producing Boolean lane; deeper CDCL integration is a separate core-engine project.

3.5 A concrete scheduling policy

The first implementation can avoid learned control altogether. Give every node a feature vector: datatype recursors present, mutable recursive parameters, existential head, polynomial degree and support, number of Boolean atoms, equality-component size, and missing theorem premises. Enable only relevant actions.

Use increasing bounded rounds. Illustrative initial limits are induction depth 2 then 4, lemma candidates 32 then 128, polynomial degree 2 then 4 then 6, retrieval batches 32 then 128, and Bernstein tree nodes 64 then 256 then 1,024. These are proposed defaults to tune, not empirically optimal constants.

A useful action priority is

$$S(a) = \frac{\widehat{\Delta\text{solved}}(a) + \eta \widehat{\Delta\text{usefulFacts}}(a)}{\varepsilon + \widehat{T}_{\text{search}}(a) + \lambda \widehat{T}_{\text{replay}}(a)} + \rho \text{age}(a).$$

The numerator rewards useful progress, not the raw number of generated terms. The denominator includes reconstruction cost: a solver that finds a huge proof faster can still lose overall. An age queue receives a fixed share of work so heuristic priorities cannot permanently starve an eligible bounded action. None of these scores affects soundness.

Proof reconstruction is itself a budgeted task. Store shared derivations as a DAG, retain theorem applications rather than duplicating expanded proof terms, and reject an oversized certificate as UNKNOWN rather than accepting it on trust. The scheduler records whether a timeout occurred in search, parsing, reconstruction, kernel checking, or export.

4 Structural search: induction that strengthens the right statement

4.1 Why generalization is a first-class action

Consider the following custom list functions, written mathematically as equations:

$$\begin{aligned} \text{app}([], y) &= y, & \text{app}(h :: t, y) &= h :: \text{app}(t, y), \\ \text{rev}([]) &= [], & \text{rev}(h :: t) &= \text{app}(\text{rev}(t), [h]), \\ \text{qrev}([], a) &= a, & \text{qrev}(h :: t, a) &= \text{qrev}(t, h :: a). \end{aligned}$$

The desired theorem is

$$\text{qrev}(x, a) = \text{app}(\text{rev}(x), a). \quad (1)$$

A naive induction on x with a fixed gives a hypothesis only at accumulator a . The recursive call needs it at $h :: a$. The correct motive is

$$P(x) \equiv \forall a, \quad \text{qrev}(x, a) = \text{app}(\text{rev}(x), a).$$

This is a change to the proof plan, not an extra ground rewrite.

The prototype reads recursive defining equations and records positions whose arguments change in a recursive call. It identifies position 1, using zero-based indexing, of `qrev` as a changing parameter. When the current goal contains `qrev(x, a)`, it proposes generalizing a before induction on x . This is a small, explicit algorithm, not an appeal to an unspecified language model.

4.2 The general Lean algorithm

For each recursive application in the goal or relevant hypotheses, inspect its equation theorems and recursive-call arguments. Score an induction variable by how often it controls recursive descent and how much unfolding its constructor cases would unlock. Collect variables occurring in nonstructural arguments that mutate in those recursive calls.

Then close the generalization set under dependent-context requirements. For example, if $v : \text{Vector } \alpha n$ and n must be generalized, a declaration depending on v or n cannot be left ill-typed in the old context. Revert dependent declarations in a valid order, form the stronger motive, and use Lean’s recursor/induction infrastructure to create the cases. Reintroduce the reverted declarations inside each case.

The planner should try a small lattice of motives rather than always generalize everything: no generalization; mutation-driven variables; dependency closure of that set; and a bounded wider set. Too much generalization can produce unnecessarily strong and false auxiliary conjectures or unhelpfully large hypotheses. Each motive is accepted only if its ordinary Lean proof closes.

For a structurally recursive list function, the recursive hypothesis is permitted only at the immediate tail in this simple planner. For other datatypes, use the actual recursive fields supplied by the recursor. For well-founded recursion, the proof plan must provide the relation and decrease obligations. For mutually recursive definitions, use the appropriate mutual principle; two unsupported circular conjectures are not a substitute.

4.3 The step case, with every dependency visible

The induction step for (1) becomes

$$\begin{aligned}
 \text{qrev}(h :: t, a) &= \text{qrev}(t, h :: a) && \text{definition} \\
 &= \text{app}(\text{rev}(t), h :: a) && \text{generalized IH} \\
 &= \text{app}(\text{app}(\text{rev}(t), [h]), a) && \text{append associativity and definitions} \\
 &= \text{app}(\text{rev}(h :: t), a) && \text{definition.}
 \end{aligned}$$

The middle step needs an append law, not just the right induction hypothesis. The two missing ingredients are therefore separable: *strengthen the motive* and *supply a proved auxiliary lemma*. The prototype’s ablation deliberately tests both.

4.4 Schema-driven lemma synthesis

The implemented lemma generator reads the function signature. For each homogeneous binary list operator f , it proposes the schemas

$$f(x, []) = x, \quad f(f(x, y), z) = f(x, f(y, z)).$$

It additionally has an explicit reversal/append anti-homomorphism schema:

$$\text{rev}(\text{app}(x, y)) = \text{app}(\text{rev}(y), \text{rev}(x)).$$

The generator does not know in advance which homogeneous binary operator is associative. It proposes the schemas for both app and qrev . Finite evaluation rejects the false qrev candidates. Surviving conjectures are then proved by constructor-complete induction and admitted in dependency order.

The finite filter is never a prover. For example, qrev ’s right-identity candidate is false at $x = [0, 1]$. Its associativity candidate is false at $x = [0]$, $y = [1]$, $z = []$: the two sides are $[1, 0]$ and $[0, 1]$. Finding these examples safely saves proof-search work; failing to find an example establishes nothing.

A richer implementation can derive candidate schemas from stuck normal forms. Typed anti-unification replaces corresponding differing subterms by shared fresh variables, preserving repeated-occurrence constraints and types. Orient candidates by a well-founded term order or a measured simplification benefit. Restrict size, new variables, and nested function depth, and put the candidate in the conjecture queue. This extension is proposed; the implemented generator uses the explicit schemas above.

4.5 Proof traces, not search replay

The structural prototype has a small typed term language with list and element sorts. Its producer performs bounded rewriting using definitions, earlier proved lemmas, and the current induction hypothesis. It records each rewrite’s rule name, subterm path, and explicit substitution.

The checker does not rerun matching or normalization. It checks the supplied substitution and sorts, confirms that the focused subterm is exactly the instantiated left side, performs the stated replacement, and compares the final terms. It reconstructs the base and cons cases from the original statement and creates the legal induction hypothesis itself. The certificate cannot supply an arbitrary stronger IH.

Bundles are checked in topological order from definitions alone. Definition names and the local IH name are reserved; duplicates and forward dependencies are rejected. An explicitly required target must really be proved, rather than disappearing when a record is relabeled `UNKNOWN`. This last condition prevents a subtle vacuous-success bug in bundle interfaces.

Theorem 4.1 (Soundness of the structural certificate scheme). *For well-sorted equations in the prototype’s list signature, suppose all defining equations have their ordinary list interpretation. If the checker accepts a required theorem using only previously checked lemmas, then that universally quantified equation holds in this interpretation.*

Proof. Each accepted rewrite is an instance of a valid defining equation, an earlier valid universal lemma, or the restricted induction hypothesis. Replacing equal subterms preserves equality in a typed context. Consequently, matching final normal forms establishes each checked case equation. The base and cons cases are exactly the cases of list induction. Generalized parameters are universally quantified in the motive, so their instances in the step are licensed; fixed parameters cannot be instantiated arbitrarily. Induction gives the theorem. A topological induction on the lemma bundle establishes the validity of every admitted earlier lemma. $\square$

This theorem is a mathematical argument about the checker scheme, not a machine-checked proof about the Python implementation. Porting the term language, checker, and soundness theorem to Lean would establish a stronger assurance level.

4.6 Where this reaches beyond ground reasoning

The complete implemented configuration proves append right identity, append associativity, reversal over append, (1), and reversal involution. The accumulator theorem fails in the prototype if either generalization or the proved append lemmas are disabled. This demonstrates an actual interaction between proof-planning mechanisms.

It does *not* establish that stock grind failed on a compiled Lean version of these examples. Definitions, available library lemmas, settings, and compiler versions matter. The custom-function versions are included as candidate separating benchmarks so that this question can be tested without accidentally retrieving a pre-existing theorem about the same custom function.

5 Nonlinear arithmetic: exact search certificates, not floating-point proofs

5.1 The gap to target

Polynomial equality and polynomial inequality are different problems. An algebraic normalizer can prove an identity, but nonnegativity also needs order facts. A useful extension therefore searches for a decomposition whose terms have known signs, then asks a small checker to confirm the identity exactly.

The first stage should still try inexpensive existing normalization, positivity, and `nlinarith`. Only a stalled, appropriately typed polynomial component receives a more expensive certificate search. The prototype implements three complementary routes: exact rational quadratic decomposition, a bounded sparse cone with equality multipliers, and Bernstein certificates on boxes. They share one exact sparse-polynomial representation.

5.2 The certificate language

Let $g_1, \dots, g_m \in \mathbb{Q}[X]$ be hypotheses known nonnegative and let $h_1, \dots, h_r \in \mathbb{Q}[X]$ be hypotheses known equal to zero. To prove $p \geq 0$, accept an identity

$$p = \sum_{k=1}^K c_k s_k^2 \prod_{i \in I_k} g_i + \sum_{j=1}^r t_j h_j, \quad c_k \in \mathbb{Q}_{\geq 0}, \quad s_k, t_j \in \mathbb{Q}[X]. \quad (2)$$

The implementation stores sparse exponent vectors and rational coefficients as exact strings. Each term contains its nonnegative rational weight, a polynomial to square, and indices of input inequalities. Equality terms contain an input-equation index and an unrestricted polynomial multiplier. The checker is given p , the g_i , and the h_j from outside the certificate; the solver is not allowed to choose an easier theorem statement.

Theorem 5.1 (Cone-certificate soundness). *In any ordered field containing the interpreted rational coefficients, (2), together with $g_i \geq 0$ and $h_j = 0$, implies $p \geq 0$.*

Proof. Squares and the c_k are nonnegative. A finite product of nonnegative factors is nonnegative, and finite sums preserve nonnegativity. Every $t_j h_j$ is zero. Substituting the checked polynomial identity proves the claim. $\square$

The important implementation consequence is a small trust boundary: the search may use heuristic algebra and numerical optimization, but acceptance needs only exact rational arithmetic and a sign argument. A Lean implementation must additionally prove that the reified input polynomials evaluate to the user's actual expressions.

5.3 An exact and complete quadratic subsolver

For a rational polynomial $p(x)$ of degree at most two, construct its symmetric augmented matrix Q so that

$$p(x) = z^T Q z, \quad z = (1, x_1, \dots, x_n)^T.$$

The constant is Q_{00} , linear coefficients are $2Q_{0i}$, diagonal quadratic coefficients are Q_{ii} , and mixed coefficients are $2Q_{ij}$. Search by exact symmetric elimination. A positive diagonal pivot a in a block

$$Q = \begin{pmatrix} a & b^T \\ b & C \end{pmatrix}$$

gives the identity

$$z^T Q z = a (z_0 + a^{-1} b^T z')^2 + z'^T (C - a^{-1} b b^T) z'.$$

All computations remain rational; square roots are unnecessary. A negative diagonal rejects positive semidefiniteness. If all remaining diagonals vanish but some off-diagonal entry is nonzero, a two-variable restriction takes a negative value, so the remainder is not positive semidefinite. An entirely zero remainder finishes the decomposition.

Proposition 5.2 (Quadratic scope). *For rational p of degree at most two, $p(x) \geq 0$ for every $x \in \mathbb{R}^n$ if and only if its augmented matrix Q is positive semidefinite. The exact pivot algorithm above decides this property and, in the positive case, returns a rational weighted sum of affine squares.*

Proof. If Q is positive semidefinite, substitute $z = (1, x)$. Conversely, for $z_0 \neq 0$, $z^T Q z = z_0^2 p(z'/z_0) \geq 0$; continuity extends the inequality to $z_0 = 0$. At a positive pivot, completing the square makes positive semidefiniteness equivalent to that of the Schur complement. The negative-diagonal and zero-diagonal/off-diagonal rejection conditions are necessary, and each pivot reduces dimension. Induction proves both termination and the rational decomposition claim. $\square$

This completeness statement is for *unconstrained quadratic nonnegativity over the reals*, not for arbitrary constrained polynomial inequalities. The code sends its resulting decomposition through the same independent sparse checker used for general cone certificates.

5.4 A concrete sparse-cone search

The implemented general search is intentionally more modest than a full semidefinite solver. For a degree budget D , generate monomials m up to degree $\lfloor D/2 \rfloor$, squares m^2 , and binomial squares $(m_i \pm m_j)^2$. Multiply monomial squares by one input inequality or by a product of two distinct input inequalities, retaining only terms of degree at most D . Generate signed monomial multiples of every equality hypothesis, also under the degree bound.

Expand these candidate columns in a common monomial basis, obtaining an exact rational matrix A and the target coefficient vector b . Search for

$$A\lambda = b, \quad \lambda \geq 0.$$

Signed equality columns allow unrestricted equality multipliers without requiring unrestricted variables in this LP. A simple objective prefers small total weight and slightly penalizes high degree. The prototype uses SciPy's `linprog(method='highs')` for support selection and SymPy's exact matrix routines for reconstruction [20, 21].

The acceptance pipeline is precise:

```
build exact rational A, b and semantic descriptors for columns
solve a floating-point LP to propose a sparse support S
solve A[:, S] * weights = b again over the rationals
instantiate any free parameters conservatively (prototype: zero)
reject if a weight is negative, irrational, or inconsistent
construct a cone/ideal certificate from the exact weights
accept only if the separate sparse checker confirms the identity
```

The numerical support threshold in the prototype is 10^{-8} . That number has *no logical meaning*: dropping a genuinely needed column can only make exact reconstruction fail. Numerical infeasibility, overflow, an inadequate support, or a poor choice of free parameters returns UNKNOWN. In particular, the implementation is not complete even for every feasible instance of its finite LP cone, because its support and free-parameter heuristics can miss an exact feasible solution.

A later version can use exact LP reconstruction with multiple supports, or search a full Gram-matrix semidefinite formulation. The latter still needs exact rational recovery and a checked PSD decomposition. A tiny floating-point residual is not an identity, and a numerically almost positive-semidefinite matrix is not a certificate.

5.5 Concrete identities returned by the design

The elementary identity

$$x^2 + y^2 - 2xy = (x - y)^2$$

illustrates the basic certificate. Higher-degree structure is also captured:

$$x^4 + y^4 - 2x^2y^2 = (x^2 - y^2)^2.$$

For three variables,

$$x^2 + y^2 + z^2 - xy - yz - zx = \frac{1}{2}(x - y)^2 + \frac{1}{2}(y - z)^2 + \frac{1}{2}(z - x)^2.$$

Under $x + y = 1$, an equality multiplier becomes useful:

$$x^2 + y^2 - \frac{1}{2} = \frac{1}{2}(x - y)^2 + \frac{1}{2}(x + y - 1)(x + y + 1).$$

The multiplier is not required to be nonnegative because its factor is zero. Under $0 \leq x \leq 1$, the target $x - x^2$ is simply $x(1 - x)$. These examples are not claimed to defeat `nlinarith`; they are transparent validation cases for the certificate interface.

5.6 Bernstein certificates on rational boxes

Let $B = \prod_{i=1}^n [a_i, b_i]$ with rational $a_i < b_i$. Substitute $x_i = a_i + (b_i - a_i)t_i$ to get $q(t) = \sum_{\alpha} c_{\alpha} t^{\alpha}$ on $[0, 1]^n$. For coordinate degree bounds d_i , define

$$B_{\beta, d}(t) = \prod_i \binom{d_i}{\beta_i} t_i^{\beta_i} (1 - t_i)^{d_i - \beta_i}.$$

The Bernstein coefficients are

$$b_{\beta} = \sum_{\alpha \leq \beta} c_{\alpha} \prod_i \frac{\binom{\beta_i}{\alpha_i}}{\binom{d_i}{\alpha_i}}. \quad (3)$$

The denominator is well defined because only actual monomial exponents $\alpha_i \leq d_i$ occur.

Lemma 5.3 (Power-to-Bernstein conversion). *For $0 \leq a \leq d$,*

$$t^a = \sum_{j=a}^d \frac{\binom{j}{a}}{\binom{d}{a}} \binom{d}{j} t^j (1 - t)^{d-j}.$$

Proof. Use $\binom{j}{a} \binom{d}{j} / \binom{d}{a} = \binom{d-a}{j-a}$, factor out t^a , and apply the binomial theorem to $(t + (1 - t))^{d-a} = 1$. $\square$

Applying this identity in every coordinate proves (3). Each $B_{\beta, d}$ is nonnegative on the unit cube and their sum is one. Thus $b_{\beta} \geq 0$ for every β proves $q \geq 0$ on the whole box.

If some coefficient is negative, subdivide at a rational interior cut and repeat. The prototype bisects axes round-robin. Its certificate is a tree: a leaf lists exact nonnegative coefficients; an internal node specifies an axis, an interior cut, and certificates for *both* closed child boxes. The checker independently recomputes the affine substitution and coefficients, and checks the entire cover. It does not trust a solver-provided selection of convenient subboxes.

Theorem 5.4 (Bernstein tree soundness). *If the covering-tree checker accepts p on B , then $p(x) \geq 0$ for every $x \in B$.*

Proof. An accepted leaf proves the inequality by the nonnegative Bernstein expansion. An accepted internal node partitions its parent box into two covering closed boxes. Induction on the finite tree propagates the two child conclusions to the parent and eventually the root. $\square$

5.7 Why complementarity matters

For $p(x) = (x - 1/3)^2 + 10^{-4}$ on $[0, 1]$, the prototype returns a tree with seven leaves, thirteen total nodes, and maximum depth six. For $(x - 1/3)^2$ without the positive margin, the same route reaches its depth limit. A dyadic subdivision never places the zero $1/3$ exactly at a cut, and a subinterval crossing this zero can retain a negative middle Bernstein coefficient. An exact square certificate, however, proves the inequality immediately.

Another deliberate test is

$$M(x, y) = x^4y^2 + x^2y^4 + 1 - 3x^2y^2.$$

It is nonnegative: AM–GM applied to x^4y^2 , x^2y^4 , and 1 gives a sum at least $3x^2y^2$. The implemented degree-six finite cone returns UNKNOWN. No assertion about numerical infeasibility can overturn the mathematical inequality. This example exposes a real limitation without requiring a claim that the prototype implements all sum-of-squares theory.

5.8 Lean reconstruction and semantics

A first Lean adapter can replay short decompositions using a proved identity, `sq_nonneg`, multiplication/addition sign lemmas, and a final order argument. The prospective `ReplayExamples.lean` illustrates this style. A scalable adapter should reify expressions, prove the checker sound once, and use a kernel-checked evaluation path for each concrete certificate.

Several translations must be guarded. Natural-number subtraction is truncated and is not integer subtraction. Rational division introduces nonzero-denominator conditions before field identities can be used indiscriminately. Casts need their homomorphism and order lemmas. Bit-vector multiplication wraps. A polynomial certificate over $\mathbb{R}$ does not prove the analogous statement over an arbitrary ordered structure lacking the required operations or laws.

Strict inequalities need explicit treatment. A checked certificate for $p \geq \varepsilon$ with a rational $\varepsilon > 0$ proves $p > 0$. A nonnegative certificate alone does not. Refutation by an identity $-1 = \text{nonnegative cone} + \text{ideal terms}$ is sound, but existence of such a certificate is not asserted for every mixture of strict, non-strict, noncompact, or transcendental constraints.

6 Boolean search with checkable theory learning

6.1 A separate lane before a core-engine rewrite

Adding SAT technology is not, by itself, a new research contribution. The useful engineering question is how to give a Lean tactic the benefits of clause learning without trusting an external solver or destroying

existing ground cooperation. The recommended first step is an independent lane for a selected, explicitly reified propositional/theory fragment. A checked consequence can then be returned as a normal Lean fact.

The executable prototype is CDCL(T) for CNF formulas whose atoms can denote integer difference constraints. It implements unit propagation, decision levels and reasons, conflict resolution, an asserting learned clause, nonchronological backtracking, and theory-conflict clauses. It deliberately uses scanning propagation rather than watched literals, retains intermediate clauses, and has no restarts or deletion. It is a correctness-oriented prototype, not a performance competitor to production SAT solvers.

6.2 Difference logic and the integer-negation trap

An atom A_i represents

$$v - u \leq c, \quad u, v, c \in \mathbb{Z}.$$

A positive literal gives a graph edge $u \rightarrow v$ of weight c . A negative literal gives the edge $v \rightarrow u$ of weight $-c - 1$, because over integers

$$\neg(v - u \leq c) \iff u - v \leq -c - 1.$$

The -1 is essential and is *not* valid for real-valued difference constraints. A real solver would need explicit strictness or another exact encoding.

The theory producer runs Bellman–Ford on asserted edges. If feasible, its potentials supply an integer model. If inconsistent, it extracts a negative cycle. Such a cycle gives a clause forbidding the simultaneous assignment of the contributing literals.

For instance, $y - x \leq 0$ and $x - y \leq -1$ sum to $0 \leq -1$. The corresponding theory lemma is $\neg A_1 \vee \neg A_2$. If both atoms are unit clauses, two resolution steps derive the empty clause.

6.3 The certificate rules

The unsatisfiability proof is a topologically ordered list with three kinds of node:

1. An input clause, which must exactly match a designated clause of the original CNF.
2. A theory clause, accompanied by signed input atoms whose edge endpoints balance and whose integer bounds sum to a negative number.
3. A binary resolvent, accompanied by two earlier parent indices and an explicitly checked complementary pivot.

The final referenced node must be empty. The checker does not run unit propagation, search, Bellman–Ford, or the conflict-analysis heuristic. For a theory node it adds integer bounds and incidence counts. This even permits a balanced negative collection of edges more general than a single simple cycle; the same cancellation proof remains valid.

Theorem 6.1 (Difference-logic clause soundness). *Let signed literals $L_1, \dots, L_k$ translate into integer bounds $v_j - u_j \leq c_j$. If every variable has equal incoming and outgoing multiplicity and $\sum_j c_j < 0$, then $\neg L_1 \vee \dots \vee \neg L_k$ is valid.*

Proof. If all L_j held, adding their bounds would cancel every variable and yield $0 \leq \sum_j c_j < 0$, a contradiction. $\square$

Theorem 6.2 (Unsatisfiability replay). *If the prototype checker accepts an empty-clause derivation from a CNF and its correctly interpreted integer difference atoms, that CNF has no satisfying integer assignment.*

Proof. Input nodes are assumptions, theory nodes are valid by the preceding theorem, and binary resolution preserves logical consequence. Induction over the proof list makes the empty clause a consequence of the assumptions, which therefore cannot all hold. $\square$

A satisfiable result is checked differently: the Boolean assignment must satisfy every input clause, and its truth values for theory atoms must match an explicitly supplied integer potential vector. This is a concrete model for this fragment, not merely an abstract Boolean assignment.

6.4 What production Lean integration must add

The prototype starts from CNF, so it has not implemented proof-producing clausification of arbitrary Lean propositions. A Lean adapter must establish the atom-to-proposition map and the soundness of any Tseitin conversion. If auxiliary variables are introduced, the translation proof must justify how a model of the original assumptions extends to them, or how the checked refutation pulls back. Proving that an unrelated CNF is unsatisfiable proves nothing about the user’s goal.

For large Boolean problems, the concrete first backend choice is CaDiCaL, already used by Lean’s `bv_decide`; reuse a pinned LRAT replay layer where its interface and trust policy fit [17]. A supported finite bit-vector leaf can be delegated directly, while integer theory clauses need their own proved interpretation and connection to the Boolean derivation. Merely sending them to a SAT solver does not establish that interpretation. Alternatively, delegate a supported SMT fragment to the `cvc5`-based LeanSMT adapter with its reconstruction route [14]. The Python resolution format is a readable reference and testing oracle, not a proposed universal replacement for LRAT or mature SMT proof formats.

A shared theory interface should return a proved clause or equality together with the local assumptions it needs. Pure model-based equalities are candidates until proved. Do not apply a textbook theory-combination theorem without checking its assumptions: finite datatypes, bit-vectors, dependent types, and overlapping arithmetic signatures complicate naive arrangements based on disjoint, stably infinite theories.

6.5 Resource behavior

The prototype’s scanning propagation costs a pass through the clause database per propagation round, and its Bellman–Ford checks are $O(VE)$ in the usual arithmetic-operation model. Resolution proof storage can grow substantially because all intermediate clauses are retained. Production improvements should include watched literals, incremental theory state with rollback, learned-clause minimization, proof DAG sharing, and certificate-size budgets.

The key expected benefit is avoiding repeated exploration of the same inconsistent combinations. It is not a promise that clause learning wins on every proof. Small arithmetic goals should remain with the low-overhead grind lane. Dispatch should consider Boolean atom count, repeated conflicts, and observed replay cost.

7 Witness synthesis with a universal certificate

7.1 Witnesses are scoped terms

For a goal $\exists y, P(x, y)$, backward proof search should construct a term $w(x)$ and then prove $P(x, w(x))$. The witness may depend only on variables in scope outside that existential. In $\forall x \exists y P(x, y)$, dependency on x is allowed; in $\exists y \forall x P(x, y)$, it is not. A synthesis engine must carry binder levels or explicit dependency sets, not infer admissible dependencies from a flattened formula.

The same rule applies to dependent types. A candidate vector must have the required length, a subtype witness must come with its predicate, and a function witness may introduce new goals under a binder. `Classical.choose` is usable when a suitable existence theorem has already been proved, not as a way to create evidence for an unsupported existential.

7.2 An implemented residue family

For fixed $a, b \in \mathbb{Z}$ and $m \in \mathbb{N}_{>0}$, consider

$$\forall n \in \mathbb{Z} \exists k \in \mathbb{Z} : \quad n \leq k < n + m \quad \wedge \quad ak \equiv b \pmod{m}. \quad (4)$$

The prototype searches for an offset d_r for each residue $0 \leq r < m$ such that

$$0 \leq d_r < m, \quad a(r + d_r) \equiv b \pmod{m}.$$

Its certificate is the finite table $(d_0, \dots, d_{m-1})$. The witness is

$$k(n) = n + d_{n \bmod m},$$

using the remainder convention $0 \leq n \bmod m < m$, including negative n .

Theorem 7.1 (Residue-table soundness). *An accepted residue table proves (4) for every integer n .*

Proof. Write $n = mq + r$ with $0 \leq r < m$. Since $0 \leq d_r < m$, $n \leq n + d_r < n + m$. Moreover,

$$a(n + d_r) - b = amq + (a(r + d_r) - b)$$

is divisible by m . Thus the constructed k has both required properties. $\square$

The universal claim comes from this quotient/remainder argument, not from testing a finite collection of n . The tests do evaluate large and negative sample integers, but only as implementation checks.

7.3 A sharper synthesis algorithm for the same family

The table exists exactly when $g = \gcd(a, m)$ divides b . Necessity follows because g divides ak and m . For sufficiency, the extended Euclidean algorithm gives $ua + vm = g$; hence $r_0 = u(b/g)$ solves $ar_0 \equiv b \pmod{m}$. All integers congruent to r_0 modulo m/g solve the congruence, and

$$k = n + ((r_0 - n) \bmod (m/g))$$

satisfies $n \leq k < n + m/g \leq n + m$.

This gives a compact future certificate: the Bézout identity, divisibility of b by g , and the remainder bounds. The current implementation intentionally uses the simpler $O(m^2)$ table search to exercise finite certificate synthesis and checking. It tests all $1 \leq m \leq 20$, $-3 \leq a \leq 5$, and $0 \leq b \leq 5$. Of these 1,080 parameter triples, 831 have verified tables and 249 fail the exact gcd divisibility condition.

7.4 General synthesis proposals

For linear equalities, use a bounded affine template and exact linear algebra. For integer congruences, use Bézout/Hermite-style certificates or finite residue partitions. For structured objects, use constructor templates and recursively generated field obligations. For a general finite term grammar, a counterexample-guided loop can alternate candidate generation and candidate refutation, with a proof attempt for the surviving candidate.

The candidate generator is untrusted. A model refuting the candidate can prune it, but a candidate surviving bounded samples is still only a candidate. The final acceptance step is a Lean proof of the instantiated predicate. Missing that distinction would turn an otherwise useful synthesis heuristic into an unsound tactic.

8 Retrieval, instantiation, and cooperation that avoid saturation blowups

8.1 Retrieve for the missing obligation

The useful retrieval query is not always the original theorem statement. After an induction step stalls at $\text{app}(\text{app}(u, v), w)$ versus $\text{app}(u, \text{app}(v, w))$, the relevant query is an associativity-shaped equation. After a conditional rewrite stalls, the relevant query is its guard. The proof graph exposes these requests naturally.

A concrete first implementation uses a typed discrimination index over theorem conclusions and principal premise heads. Normalize binder names, retain constants and relation heads, and score candidates by head compatibility, symbol overlap, term-shape similarity, premise count, and expected instantiation size. Local hypotheses and recent project lemmas receive priority. Try a small exact-matching batch before using a broader lexical or learned selector.

LeanHammer’s premise-selection pipeline is an existing implementation option for the broader retrieval tier [11]. Its role here is to propose candidates, not to bypass type checking. The selector must see only the current environment: the theorem being proved, future declarations in its file, or hidden benchmark solutions must not enter the index.

8.2 Backward premises and forward consequences share one queue

Given a candidate theorem $A_1 \rightarrow \dots \rightarrow A_k \rightarrow G$, unify its conclusion with the target in a speculative metavariable context. If this produces k unresolved premises, add an AND plan for them. Small arithmetic premises can be sent to a theory leaf; structural premises can trigger recursor search; a premise identical to a known local fact can close immediately.

Conversely, a proved premise can activate a waiting conditional equality or theorem instance. Maintain a reverse index from normalized guard heads to waiting obligations. This prevents repeated scans over every conditional lemma whenever a new fact is added. Facts carry their provenance and assumption scope so that a consequence proved in one case split is not leaked to its sibling.

8.3 E-matching memoization with the right invalidation rules

A proposed instantiation key is

(theorem id, typed substitution modulo current e-classes, scope stamp, branch stamp).

If classes merge, affected keys and pending guards must be reconsidered; otherwise the cache can suppress a newly useful instance. If the metavariable context or relevant instance resolution changes, stale entries are invalid. E-matching generations and term-growth limits remain useful safeguards rather than obstacles to remove [4].

The score for an instantiation should reward a new target-relevant equality, a guard becoming solvable, or reduced structural complexity, rather than the mere production of a larger term. Give each term family a budget so an associativity loop or recursive unfolding chain cannot consume the entire search. Within the chosen finite budget, use a fair fallback queue instead of permanently discarding lower-ranked instances.

8.4 Selective alternative forms

There is still a role for equality saturation: a single normal form may be convenient for rewriting but poor for a specialist. For example, expanded polynomials are good for identity checking while factored expressions expose nonnegative products. Rather than indiscriminately duplicate all expressions, request an alternative form only when it matches an enabled specialist's input language.

The proposed representation stores a proved equality between the current expression and each retained alternative. Extraction minimizes a cost that includes the target specialist's estimated work and the proof of the transformation. Conditional alternatives carry guard obligations. This is a consumer-driven use of existing equality machinery, not a claim that an untyped e-graph can replace Lean elaboration.

8.5 An end-to-end worked interaction

Take (1) for custom functions with no registered correctness theorem. The structural planner sees descent in the first argument and mutation in the second. It generalizes the accumulator and creates the two list-induction cases. Definitions close the base case; the step reduces to an append rearrangement after applying the generalized IH.

The stalled terms trigger an associativity-shaped lemma request. The schema queue proposes append associativity and, if useful, append right identity. A separate induction proof closes these candidates from definitions, after which their proofs are admitted locally. The original step resumes and closes. The final proof is a recursor application containing the two case proofs and a dependency on the newly proved append law.

The distinction from a tactic portfolio is concrete. A portfolio merely tries several whole-goal solvers and hopes one succeeds. Here one subsystem manufactures an obligation that another proves, and a third reuses the result inside a partially constructed proof plan. This cooperation is the central proposed implementation effort.

9 Proof acceptance and the trusted boundary

9.1 The kernel must see the original theorem

There are three separate correctness layers in an external-certificate workflow:

$$\text{Lean expressions} \longrightarrow \text{reified problem} \longrightarrow \text{certificate} \longrightarrow \text{Lean proof.}$$

The first arrow needs a proved interpretation relation; the middle search may be untrusted; the last arrow needs a proved checker or ordinary proof-term reconstruction. Correct middle-layer mathematics does not repair a bad reification of division, binders, or local assumptions.

An implementation should retain the original goal expression and check that the returned proof has exactly that type, with the intended universe instantiations and no unresolved metavariables. The final declaration must also be checked by Lean’s kernel. A successful `inferType`/unification operation inside tactic metaprogramming is part of elaboration, not a replacement for the final kernel boundary.

Theorem 9.1 (Compositional acceptance criterion). *Suppose every leaf accepted by a proof graph produces a kernel-accepted proof of its recorded proposition in its recorded context. Suppose every internal plan’s builder is a well-typed Lean proof constructor using only its checked child proofs and permitted assumptions. If a finite acyclic proof graph closes the original root and its final term is kernel checked against that root, then the original goal is proved under those assumptions and the audited axioms.*

Proof. Topologically evaluate the graph. Each leaf has the stipulated proof. At each internal node, well-typed construction from already checked children yields its claimed proof in the correct context. The root therefore has the required type, which the final kernel check confirms. $\square$

This result deliberately locates all trust in the accepted proof rules and their implementation, not in the scheduler, retrieval model, optimizer, or heuristic candidate generator.

9.2 Axiom auditing must follow the actual Lean version

The validation documentation distinguishes ordinary proof checking from native evaluation. In particular, since Lean 4.29.0, native computations used by relevant tactics may be represented by per-computation axioms rather than the older single `Lean.trustCompiler` constant [19]. A blacklist containing only that historical name is therefore insufficient.

For the strongest default mode, use ordinary proof terms or kernel-reduced checker theorems, and audit the resulting declaration with `#print axioms`. An explicit allowlist can permit the project’s chosen standard axioms, such as propositional extensionality, choice, and quotient soundness where appropriate, while excluding `sorryAx` and unapproved native-computation axioms. An optional faster trust profile must be visible to the user and benchmarked separately.

The included Lean file contains handwritten reconstruction examples and axiom-print commands, but it was not compiled. The Python code does not generate Lean axioms and must never be connected to the tactic by an “external solver says true” axiom.

9.3 Malformed or hostile artifacts

Certificate inputs should be parsed with explicit limits: total bytes, nodes, polynomial monomials, exponent size, rational numerator/denominator size, recursion depth, and referenced-atom count. Reject negative exponents, zero denominators, nonexistent input indices, forward proof references, and malformed scopes before expensive work. The small Python checker validates many structural conditions but is not presented as a hardened hostile-input service.

Resource exhaustion is not a logical failure. A timeout or out-of-memory event must return a recoverable status, not trigger a fallback that trusts the search output. Similarly, a certificate can be mathematically valid yet impractical to reconstruct; record this separately to improve dispatch decisions.

9.4 Provenance, proof minimization, and export

Every accepted fact records the input hypotheses and earlier lemmas on which it depends. Export can then remove unused facts, inline short derivations, and preserve shared long derivations. A deterministic replay command should avoid expensive discovery and use the proved lemma names, reconstructed witnesses, and checked certificates directly.

A proposed `forge?` command reports the successful strategy and emits such a replay script. An illustrative report could say that it chose induction on a particular variable, generalized one parameter, and used one synthesized lemma. That would be an explanation of proof structure, not a transcript of private heuristic deliberation or an unsupported claim of an optimal proof.

10 Executed experiments and their limitations

10.1 Environment and reproducibility

The recorded run used Python 3.13.5, SymPy 1.14.0, SciPy 1.17.0, and NumPy 2.3.5 on the supplied Linux environment. The deterministic experiment seed is 20260914. The numerical and symbolic libraries are used only by producers; `scripts/verify_artifacts.py` imports standard-library-only checking modules.

The final unit-test run is **64 passed**. Parametrized tests contain additional inner-loop cases, so 64 is a test-runner count, not the total number of mathematical instances. The experiment runner records 1,494 rows, including failed searches and ablation rows, and stores 1,238 successful proof/model bundles. Replaying all stored bundles succeeds. These are synthetic algorithm experiments, *not* a Lean or grind benchmark.

10.2 Results by family

Family	Cases	Recorded outcome
Sparse cone search	49	47 exact certificates; 2 UNKNOWN results. Forty cases are manufactured from the search's own certificate basis.
Rational quadratics	40	40 exact weighted-square certificates for manufactured rational nonnegative quadratics.
Bernstein boxes	13	12 covering certificates; the non-dyadic zero example remains UNKNOWN at the chosen depth.
Structural proof targets	2	Accumulator reversal and reversal involution proved in the typed object logic.
Auxiliary-lemma batch	5 candidates	3 proved lemmas; 2 false candidates rejected by finite counterexamples. Saved as one checked bundle.
Generalization ablation	4 configurations	Only the configuration with both proved lemmas and generalization proves the accumulator theorem.
Random Boolean/difference logic	300	189 unsatisfiable and 111 satisfiable; every answer agrees with an independent exhaustive oracle.
Pigeonhole CNFs	5	All five unsatisfiable, with replayable resolution proofs.
Residue witnesses	1,080	831 verified witness tables; 249 impossible congruence parameter triples by the exact gcd condition.

Do not divide 1,238 by 1,494 and advertise an overall theorem-proving accuracy. The denominator mixes proof searches, model searches, a lemma batch, deliberately false formulas, and overlapping ablation runs.

The manufactured families are useful for debugging and mechanism validation but favor the implemented methods by construction.

10.3 The structural ablation

Proved append lemmas	Generalize accumulator	Outcome
No	No	UNKNOWN
No	Yes	UNKNOWN
Yes	No	UNKNOWN
Yes	Yes	PROVED

This is stronger evidence of mechanism than merely demonstrating a proof with all features enabled. The same goal and rewriting engine are used in all four configurations. The result shows why a blanket “try induction” addition would miss an important part of the design.

10.4 Boolean certificates and independent oracle checks

For the randomized experiments, the number of theory atoms is at most six and the variable graph has three integer vertices. This small size permits an independent oracle: enumerate Boolean assignments satisfying the clauses and use Floyd–Warshall to check difference-constraint consistency. The producer instead uses CDCL and Bellman–Ford. All 300 recorded cases agree. A separate randomized suite inside pytest uses different seeds and also performs 300 such checks; the published experiment table counts only the recorded run.

The pigeonhole examples assign p pigeons to $p - 1$ holes with at least one hole per pigeon and at most one pigeon per hole. Their proof statistics are:

Pigeons / holes	Decisions	Conflicts	Resolution nodes	Nonchronological jumps
2 / 1	0	1	2	0
3 / 2	1	2	10	0
4 / 3	8	9	45	0
5 / 4	30	31	160	0
6 / 5	89	89	513	1

The small number of nonchronological jumps in these examples is reported rather than hidden. This is not evidence that the particular branch heuristic is competitive on industrial SAT instances.

10.5 Timing and size measurements

The CSV and JSON files record per-row search time, separate replay time, and serialized statement-plus-certificate size. The field is named `certificate_bytes` for convenience, but includes the accompanying statement in that record. Timings are one in-process pass, not a repeated statistical study. Producer timings already include their internal acceptance checks; the separately timed replay repeats checking.

For scale only, the recorded total search times were approximately 0.943 seconds for the 49 cone cases, 0.414 seconds for 40 quadratics, 0.287 seconds for 13 Bernstein cases, and 0.106 seconds for the five pigeonhole

cases. These values exclude problem-generation work outside the timed call and must not be compared to a future cold-process Lean run. Structural bundle replay also checks earlier lemmas, whereas an individual structural search may reuse them, so those two columns do not measure identical workloads.

10.6 Mutation tests and negative controls

The test suite corrupts rational weights, polynomial coefficients, hypothesis indices, Bernstein split locations and coverage, induction generalization data, constructor-case coverage, theorem order, theorem status, theory-cycle evidence, resolution pivots, and the proof root. The checkers reject these corruptions. Boundary tests include zero polynomials, empty CNFs, empty clauses, tautologies, negative witness inputs, and modulus one.

These tests do not prove the absence of all bugs. They are targeted attempts to falsify the checker implementations, combined with independent algorithms for selected fragments. A Lean formalization of the checker semantics, property-based malformed-input generation, and an external audit remain appropriate next assurance steps.

10.7 What the evidence does and does not establish

The experiments establish that the supplied algorithms execute, discover the described small certificates, and can replay their outputs independently of numerical search. They demonstrate an induction-generalization interaction, exact reconstruction from a numerical support proposal, a true inequality outside one bounded search, clause-learning proof production, and universal witness construction supported by a finite certificate argument.

They do not establish an end-to-end proof-rate improvement, a production-quality Lean plugin, superior asymptotic performance to existing engines, or a universal procedure for nonlinear arithmetic or induction. The absence of a Lean executable is an execution limitation of this session, not a mathematical objection to implementing the design.

11 The real Lean evaluation that should decide success

11.1 Strong baselines and controlled inputs

The primary comparison must include stock grind, grind with appropriate explicitly supplied lemmas and tuned limits, a cheap portfolio of existing tactics, Aesop, appropriate arithmetic specialists, and compatible LeanHammer/LeanSMT configurations. Giving Forge an associativity lemma while withholding the same available theorem from a premise-aware baseline measures a data-access difference, not necessarily a search improvement.

There should be two separate tracks. The *cold-knowledge track* allows only the actual pre-theorem environment and tests missing-lemma synthesis. The *equal-premise track* gives all methods the same supplied premises and tests proof search and cooperation. A third ablation can give a hand-written correct induction motive to every eligible method, isolating motive selection from leaf solving.

11.2 Benchmark families

Use held-out, dependency-clean declarations from multiple projects, supplemented by generated families with known semantics. Essential categories are custom recursive programs with accumulator or dependent-index invariants; existential arithmetic with scope-sensitive witnesses; semialgebraic inequalities with side conditions; Boolean-heavy verification conditions; higher-order library reasoning; and easy ground goals where extra machinery could regress.

For recursive programs, include trees, mutually recursive syntax, nested recursion, and dependent vectors rather than only append/reverse. For arithmetic, include rationals with denominator side conditions, integer versus real distinctions, zero-margin inequalities, and unsupported transcendental expressions that should return UNKNOWN. For Boolean search, include theory-heavy and theory-light formulas, not just pigeonholes.

Do not train or tune on theorem statements whose proofs or near-duplicates occur in the held-out evaluation. Respect declaration order when building retrieval indexes. For synthesized lemmas, record whether an equivalent library lemma was already available, and compare against a retrieval baseline that can find it.

11.3 Measurements and acceptance criteria

Report solved goals at several fixed wall-clock and heartbeat budgets, median and tail latency, peak memory, kernel-checking time, certificate bytes, resulting proof-term size, and the count of failed reconstructions. Keep deterministic replay success as a separate metric. Any accepted false theorem or unapproved axiom is a release-blocking failure, not a minor accuracy regression.

For claims of a substantial capability gain, a reasonable *proposed acceptance target* is a statistically supported improvement on the hard held-out set while maintaining near-baseline latency on easy goals, with results broken down by family. A specific percentage should be selected before evaluation rather than invented after observing the results. Protected mode must retain baseline successes under its explicitly larger resource allowance; equal-total-budget mode must report regressions as well as gains.

Use paired comparisons on the same goals and confidence intervals over appropriate sampling units, preferably projects or theorem clusters when examples are correlated. Run multiple repetitions for timing, separate import/startup cost, and keep compiler/library revisions fixed. These are evaluation requirements, not completed measurements in this archive.

11.4 The supplied harness

`scripts/run_lean_bench.py` runs a small set of candidate separating examples in an already built Mathlib/Lake project. It preflights imports, records the actual Lean version, runs each tactic in a fresh process with explicit time and heartbeat bounds, preserves source and diagnostics, and distinguishes setup failure, timeout, rejection, and accepted elaboration. It can accept additional installed tactic strategies and module imports.

The harness has not been run against Lean here, and its five cases are not the full evaluation above. Its purpose is to make the next experiment concrete rather than leave a bare recommendation to “benchmark it.” The proposed `forge` command is not included as an executable Lean tactic and must not be passed to the harness until an implementation is actually installed.

12 An implementable development plan

12.1 Module boundaries and order of work

Stage	Concrete work	Exit condition
1. Baseline harness	Pin toolchains; record goal snapshots; implement protected runs and deterministic result logging.	Reproduce baseline results without changing proofs or axioms.
2. Structural planner	Recursor selection, dependency-closed generalization, saved-state AND/OR search, small lemma schemas.	Custom accumulator and tree examples compile and replay without external trust.
3. Polynomial replay	Reified syntax, evaluation relation, cone/ideal soundness, exact quadratic decomposition, bounded search adapter.	All generated valid certificates replay; mutated ones fail; original expressions are tied to reification.
4. Witness engine	Affine templates, residue/Bézout certificates, explicit binder dependencies, constructor witnesses.	Infinite-domain witness families have kernel-checked correctness proofs.
5. Box certificates	Bernstein conversion theorem, rational interval splits, full-cover checker, depth/node budgets.	Zero-margin failures are handled as UNKNOWN; positive-margin examples replay.
6. Boolean adapter	Certified atom map, CNF translation, theory clauses, production proof-format integration.	End-to-end original Lean propositions, not only abstract CNFs, are checked.
7. Shared scheduling	Retrieval requests, guarded facts, scope-safe caches, proof-cost model, minimization/export.	Measured benefit over a simple whole-goal portfolio.
8. Evaluation and hardening	Dependency-clean corpora, adversarial certificates, axiom policies, compatibility and regression testing.	Publish paired results, failure analysis, and a reproducible release.

This sequence deliberately postpones a wholesale `grind` engine fork. Structural search and ordinary theorem reconstruction can be implemented outside the core first. Only profiling and controlled comparisons should justify invasive changes to its saturation or branch machinery.

12.2 Concrete implementation assets

Use Lean’s own metaprogramming and recursor infrastructure for goal manipulation, and an Aesop-style search representation where it reduces implementation risk [10]. Use Mathlib’s algebra and order lemmas for initial replay, then replace repeated tactic expansion with proved reflective checkers where measurements justify it. Optional premise selection and higher-order reasoning should use the existing LeanHammer/Lean-auto/Duper ecosystem behind versioned adapters [11, 13, 12].

For the currently executable search path, use the pinned SymPy and SciPy versions in `requirements.txt`. The independent Python polynomial checker uses `fractions.Fraction`, dictionaries, binomial coefficients, and explicit finite loops. The SAT/difference and structural checkers also use only the standard library. This separation is useful both for testing and as a blueprint for the eventual Lean trusted boundary.

An SMT backend should expose a proof or a reconstructible explanation, not only a Boolean answer. A future numerical semidefinite backend should expose enough data for exact rational recovery. The library choice

is secondary to the certificate contract: replacing a solver should not require changing what constitutes an accepted proof.

12.3 Proposed user-facing behavior

A small initial interface could be:

```
forge                -- protected baseline plus bounded extensions
forge?              -- discover and print a deterministic replay
forge only [lemma1, lemma2] -- explicit premise scope
forge (structural := false) -- disable structural goal creation
forge (certificates := true) -- require the selected certificate policy
```

These commands are a *proposed syntax*, not commands implemented by the archive. Budget and trust options should have explicit documented meanings. A global “aggressiveness” knob is less useful than separate structural depth, theorem-instantiation, polynomial-degree, Boolean-conflict, and replay-size limits.

Failures should explain the remaining obstruction at the level of the proof: missing induction generalization, unresolved guard, no certificate found within a specified degree, unsupported atom translation, witness template exhaustion, or reconstruction budget exceeded. A candidate model of a relaxation should be labeled as such. Do not report “counterexample” unless its interpretation really refutes the original proposition.

12.4 The most important engineering risks

The largest soundness risk is mistranslation at the Lean/certificate boundary, particularly with local assumptions, casts, dependent binders, and strict inequalities. The largest performance risk is generating too many candidates whose proofs are expensive to reconstruct. The largest evaluation risk is comparing against an artificially weak premise set or leaking target theorems into retrieval.

The mitigation is systematic: prove the reification relation; keep branch-local state and explicit dependency graphs; record search and replay separately; test bad certificates; and compare against strong existing tools with identical available knowledge. The architecture should make these controls easy to inspect rather than hide them behind a monolithic tactic.

13 Conclusions

A significantly more capable successor to grind should be built by expanding proof planning around its existing reasoning strengths. The highest-value addition is a structural layer that can change the theorem it inducts on, prove missing lemmas, and construct witnesses. Exact nonlinear and Boolean/theory certificates add complementary capabilities while retaining a narrow proof-acceptance boundary.

The supplied experiments make several parts of this proposal concrete. Generalization and auxiliary lemmas are jointly necessary in an executed structural ablation. Numerical support selection produces exact rational certificates that a separate checker verifies. Bernstein trees certify whole boxes and honestly fail on a troublesome zero. CDCL(T) emits explicit theory and resolution proofs whose conclusions match independent small-instance oracles. Residue tables certify witnesses for every integer input through a proved mathematical reduction to finitely many residues.

The resulting proposal is therefore more than a list of tactics to try, but less than a falsely advertised finished Lean prover. The next decisive step is the Lean implementation of these interfaces and reconstruction theorems, followed by the controlled evaluation specified above. The algorithms, negative cases, executable artifacts, and acceptance criteria are provided so that this step is technically well defined.

A Archive map and reproduction commands

The top-level archive directory is `forge-research`. Its contents are:

<code>article/forge.tex, forge.pdf</code>	Article source and compiled PDF
<code>prototype/forge_cert/</code>	Executable producers and small checkers
<code>poly.py</code>	Exact sparse polynomial / Bernstein checking
<code>polynomial_search.py</code>	Quadratic, finite-cone, and box search
<code>induction.py</code>	Typed induction, lemmas, explicit trace replay
<code>cdclt.py</code>	CDCL(T), resolution and difference checking
<code>witness.py</code>	Residue synthesis and table checking
<code>tests/test_certificates.py</code>	Positive, negative, oracle, mutation tests
<code>scripts/run_experiments.py</code>	Deterministic synthetic experiments
<code>scripts/verify_artifacts.py</code>	Search-independent saved-artifact replay
<code>scripts/run_lean_bench.py</code>	Unexecuted Lean comparison harness
<code>lean/ReplayExamples.lean</code>	Uncompiled prospective proof reconstructions
<code>lean/INTEGRATION.md</code>	Module contracts and version/trust boundaries
<code>lean/benchmarks.json</code>	Candidate separating Lean statements
<code>results/</code>	Raw results, saved certificates, test logs
<code>sources/</code>	Source and status notes
<code>README.md, requirements.txt, Makefile, LICENSE</code>	

From an unpacked directory with a compatible Python environment:

```
python -m pip install -r requirements.txt
python -m pytest -q
python scripts/run_experiments.py
python scripts/verify_artifacts.py
make pdf
```

The saved artifact replay itself needs no NumPy, SciPy, SymPy, or pytest. It can be run directly with the Python standard library. It verifies the recorded object-level statements; it does not verify the correspondence of those statements to an external Lean goal.

In an independently prepared, compatible Mathlib project, the next Lean commands are:

```
# This was NOT run during article preparation.
cd /path/to/existing/mathlib-project
lake env lean /path/to/forge-research/lean/ReplayExamples.lean

python /path/to/forge-research/scripts/run_lean_bench.py \
  --project /path/to/existing/mathlib-project \
  --timeout 30 --heartbeats 1000000 \
  --output /path/to/results/lean-benchmark.json
```

Those commands require a real Lean/Lake installation and built dependencies. An import or elaboration error must be resolved and recorded before any success is claimed. The article's version snapshot does not

guarantee compatibility of an arbitrary existing project.

B A minimal executable certificate example

The following uses the actual Python modules in the archive:

```
import sys
sys.path.insert(0, "prototype")
import sympy as sp
from forge_cert.polynomial_search import cone_search, sparse
from forge_cert.poly import verify_cone

x, y = sp.symbols("x y")
target = x**4 + y**4 - 2*x**2*y**2
certificate, stats = cone_search(target, (x, y), degree=4)
assert certificate is not None
assert verify_cone(sparse(target, (x, y)), [], [], certificate, 2)
print(certificate)
```

The independent acceptance step can instead read the already serialized polynomial and certificate, with no symbolic algebra library. That is precisely what `verify_artifacts.py` does for the saved run.

A structural example is:

```
from forge_cert.induction import (
    F, x, a, prove, verify, synthesize_lemmas
)
lemmas, records = synthesize_lemmas()
lhs = F("qrev", x, a)
rhs = F("app", F("rev", x), a)
assert prove(lhs, rhs, lemmas, generalize=False) is None
certificate = prove(lhs, rhs, lemmas, generalize=True)
assert certificate is not None
assert verify(lhs, rhs, certificate, lemmas)
```

For untrusted saved lemma bundles, use `verify_bundle` with required theorem names rather than trusting an externally supplied list of lemmas. The experiment verifier does so. This distinction is part of the artifact contract, not an optional optimization.

C Scope ledger

Claim	Status in this deliverable
Exact rational cone/ideal identity checking	Implemented and tested in Python; mathematical soundness argument included.
Quadratic positive-semidefinite decomposition	Implemented with exact rational pivots; tested on manufactured cases.
Bernstein subdivision and cover checking	Implemented and tested; bounded, intentionally incomplete search.

Claim	Status in this deliverable
Induction generalization and lemma schemas	Implemented for a small typed list-equation language, not arbitrary dependent Lean goals.
CDCL(T) and difference-logic proof replay	Implemented for CNF and integer difference atoms; no arbitrary Lean clausifier.
Residue witness certificates	Implemented finite table synthesis and checking; universal mathematical argument given.
Lean tactic command <i>forge</i>	Proposed, not implemented in this archive.
Lean replay examples	Authored but not compiled in this session.
Search-independent artifact replay	Executed successfully on 1,238 saved bundles.
Lean-kernel proof-rate improvement over <i>grind</i>	Not measured; a concrete controlled evaluation is specified.
Production security and scalability	Not established by the small prototype test suite.

References

- [1] Lean reference manual, *The grind tactic*. Current manual inspected 14 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [2] Lean reference manual, *Algebraic solver (commutative rings/fields)*. https://lean-lang.org/doc/reference/latest/The--grind--tactic/Algebraic-Solver-_LPAR_Commutative-Rings___Fields_RPAR_/.
- [3] Lean reference manual, *Linear arithmetic solver*. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/Linear-Arithmetic-Solver/>.
- [4] Lean reference manual, *E-matching*. https://lean-lang.org/doc/reference/latest/The--grind--tactic/E__matching/.
- [5] Lean reference manual, *Case analysis*. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/Case-Analysis/>.
- [6] Lean project, *Lean 4.34.0 release*. Published 14 September 2026; release API inspected on the same date. <https://github.com/leanprover/lean4/releases/tag/v4.34.0>.
- [7] Lean API documentation, *Lean.Meta.Tactic.Grind.Types*, including solver-extension interfaces. Inspected 14 September 2026. <https://lean-lang.org/doc/api/Lean/Meta/Tactic/Grind/Types.html>.
- [8] Lean source, *Lean/Meta/Tactic/Grind/Main.lean*; development-branch excerpt inspected 14 September 2026, blob id f0355a372e08f69d863b94ee39541a8b96d300e5. This is a file blob identifier, not a repository commit. <https://github.com/leanprover/lean4/blob/master/src/Lean/Meta/Tactic/Grind/Main.lean>.
- [9] E. Wang, S. Chess, S. Szeto, and T. Meek, *Learned Interventions Inside Lean 4’s grind*, arXiv:2607.22972v1, 2026. <https://arxiv.org/html/2607.22972v1>.
- [10] Aesop project, official repository and documentation. <https://github.com/leanprover-community/aesop>.
- [11] T. Zhu, J. Clune, J. Avigad, A. Q. Jiang, and S. Welleck, *Premise Selection for a Lean Hammer*, arXiv:2506.07477v2, 25 February 2026. <https://arxiv.org/html/2506.07477v2>.
- [12] Duper project, official repository and documentation. <https://github.com/leanprover-community/duper>.
- [13] Y. Qian, J. Clune, C. Barrett, and J. Avigad, *Lean-auto: An Interface between Lean 4 and Automated Theorem Provers*, arXiv:2505.14929, and official project documentation. <https://arxiv.org/abs/2505.14929>; <https://github.com/leanprover-community/lean-auto>.

- [14] A. Mohamed et al., *Lean-SMT: An SMT tactic for discharging proof goals in Lean*, arXiv:2505.15796v1, 2025, and official LeanSMT repository. <https://arxiv.org/html/2505.15796v1>; <https://github.com/ufmg-smite/lean-smt>.
- [15] Mathlib API, *Mathlib.Tactic.Linarith.Frontend*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Linarith/Frontend.html.
- [16] Mathlib API, *Mathlib.Tactic.Linarith.Verification*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Linarith/Verification.html.
- [17] Lean reference manual, *Tactic reference*, including `bv_decide` and decision tactics; Lean source, *Lean.Meta.Tactic.BVDecide*. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>; <https://github.com/leanprover/lean4/blob/master/src/Lean/Meta/Tactic/BVDecide.lean>.
- [18] Lean Reservoir, *marcusrossel/egg*, project README and deprecation notice. Inspected 14 September 2026. <https://reservoir.lean-lang.org/%40marcusrossel/egg>.
- [19] Lean reference manual, *Validating a Lean Proof*, including native-computation changes since Lean 4.29.0. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [20] SciPy 1.17.0 documentation, *scipy.optimize.linprog*. <https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.optimize.linprog.html>.
- [21] SymPy 1.14.0 documentation, *Matrices (linear algebra)*. <https://docs.sympy.org/latest/modules/matrices/matrices.html>.